

KNN para classificação

Olá! Neste vídeo iremos apresentar o KNN versão classificação. O modelo KNN, assim como vários outros modelos de aprendizado de máquina possuem versões para classificação e regressão, ou seja, possuem métodos diferentes para trabalhar com dados de classificação com rótulos de classe, ou com regressão onde os rótulos são números ordenáveis. Então, a diferença entre os dois é bem básica.

Irei iniciar mostrando o método de classificação e, em um vídeo posterior, irei apresentar o método de KNN para regressão, má adianto que a diferença entre os dois é bem simples.

Inicialmente iremos utilizar o Sklearn para importar o método KNN para que não seja necessário implementar o código Python por baixo dele.

Utilizaremos a ferramenta Sklearn que possui uma boa implementação do KNN. Então, iremos utilizar uma célula com o comando: `from sklearn.neighbors import KNeighborsClassifier`.

Utilizando Shift + Enter para executar essa célula, vamos ter importado o método KNN para classificação.

A primeira coisa que iremos fazer é implementar este método, porque nós importamos a classe.

Instanciaremos esse método em uma variável, então iremos utilizar "knn", minúsculo. Esse nome é uma variável, você pode utilizar outro que preferir. Eu estou utilizando "knn", minúsculo. "knn = KNeighborsClassifier", escrito da mesma forma que aparece na célula de importação.

"knn = KNeighborsClassifier (n_neighbors=3)", onde "(n_neighbors=3)" é um parâmetro.

O que isto está querendo dizer?

Para a classificação, o KNN funciona de uma maneira bem simples.

Nós temos um conjunto de dados utilizados para treinar o modelo que são os dados de treino.

E ao treinar o KNN, iremos armazenar estes dados de treino que serão comparados a novos dados posteriormente.

A cada novo dado que chega iremos calcular a distância deste novo dado.

Para cada um dos dados de treino verificaremos aqueles vizinhos mais próximos do dado de treino.

Esses vizinhos mais próximos do dado de treino são escolhidos para a classificação. Esse parâmetro, "n_neighbors=3", está sendo utilizado para informar ao KNN que cada novo elemento será comparado aos três vizinhos mais próximos a ele.

Logo, podemos alterar o parâmetro dependendo do conjunto de dados e da complexidade dele.

Então, esta técnica é bem simples porque ela compara um novo dado aos dados de treino verifica aqueles que estão mais próximos e repete a classe da maioria.

Suponha que exista as classes 0 e 1 se estamos procurando pelos três vizinhos mais próximos e dois desses vizinhos são da classe 1 e o terceiro é da classe 0 podemos dizer que esse novo dado tem a mesma classe da maioria dos vizinhos mais próximos.

Dessa forma, estaremos classificando o dado de acordo com os dados que foram armazenados anteriormente, ou seja, os dados de treino.

O modelo é bem simples, não tem nenhum segredo uma vez que aprendemos a ideia de distância o modelo passa a ser bem simples.

Podemos utilizá-lo e entender a intuição.

Uma das coisas que deixaremos bem claro é que em colunas numéricas, porém que representam classes por exemplo se o número 1 for um tipo de fruta e o número 2 for outro tipo de fruta a distância entre esses atributos não é tão interessante porque se o número 1 for uma maçã e o número 2 uma laranja e o número 3 um limão não significa que a laranja e a maçã estão mais próximas por serem 1 e 2, do que o limão que seria 1 e 3 não estariam mais distantes por isso.

Esses números não significam um valor numérico.

Então, quando tivermos colunas de atributos que possuem números, mas não representam um valor numérico o KNN não é tão interessante de ser utilizado.

Que fique bem claro sobre o KNN uma vez que o KNN lida com distância, se as colunas forem números que não representam elementos numéricos esse método não é aconselhado.

Então, ao rodar a linha: `knn = KNeighborsClassifier(n_neighbors=3)`, estamos instanciando o KNN na variável `knn`, minúsculo.

Executando esta linha temos o KNN implementado na variável `knn`, minúsculo.

Então, temos o nosso modelo já pronto tão simples quanto implementar uma variável de uma classe.

Para executar o KNN, iremos utilizar um conjunto de dados. O conjunto de dados escolhido para esta aula principalmente a parte de classificação será o conjunto de dados de frutas.

Iremos capturá-lo com os comandos a seguir.

Utilizaremos: `data = pd.read_table('fruit_data_with_colors.txt')`. Este comando não funcionará agora porque o Pandas ainda não foi importado. Podemos importá-lo na célula na qual o KNN foi importado.

Podemos utilizar: `"import pandas as pd"`

Executamos a célula para importar o Pandas antes. Ao executar a célula, na qual chamamos a leitura do arquivo conseguiremos importá-lo sem nenhum problema.

Teremos o conjunto de dados como também podemos executar a célula com o nome `"data"` que seria a variável dos dados e teremos os dados armazenados.

Então, para efetuar algum tipo de predição ou técnica de aprendizado de máquina em cima dos dados iremos usar apenas os atributos dessas frutas que seriam as colunas numéricas que temos a massa, a altura, a largura e a cor.

E iremos armazenar também o rótulo ou label dessas frutas, que está na coluna `fruitlabel` composta por números.

Iremos criar duas variáveis para separar os atributos do rótulo. Iremos utilizar a variável "X", para armazenar os atributos. Ficando: `X = data[['mass', 'height', 'width', 'color_score']]`.

Iremos quebrar a linha e na linha abaixo dentro da mesma célula, iremos utilizar: `y = data['fruit_label']`.

E temos o rótulo das frutas sendo armazenado na variável `y`, minúsculo.

Ao executar essa linha, teremos, na variável `X`, apenas os atributos de massa, altura, largura e cor.

E, na variável `y`, teremos os rótulos de cada uma dessas frutas. Então, agora que temos rótulos e atributos ou vice-versa iremos utilizar a função de divisão entre treino e teste para separar o nosso conjunto de dados em treino, que será utilizado para construir o modelo e em teste, que será utilizado para avaliar o modelo. Nós separamos em treino e teste para que o modelo não fique viciado nos dados.

O que isso significa?

Se treinarmos utilizando todos os dados e testarmos em cima dos mesmos o modelo irá aprender em cima daquele conjunto de dados. As previsões podem não ser consistentes para dados futuros, que ainda não foram vistos pelo modelo. Então, geralmente, separamos esses dados e avaliamos o modelo em um conjunto de dados diferente dos dados utilizados para o treinamento.

Para importar a função "train-test split" iremos utilizar o seguinte comando: `from sklearn.model_selection import train_test_split`.

Ao executar este comando teremos importado a divisão em treino e teste. Com essa função, nós podemos separar o conjunto de dados em dois conjuntos de dados diferentes que são os dados de treino e os de teste. Na linha seguinte, após executar a célula que importa a divisão em treino e teste iremos escrever o seguinte: `"X_train,"`, que é onde ficarão os atributos de treino `"X_test,"`, que usaremos para armazenar os atributos de teste `"y_train,"`, que é onde armazenaremos os rótulos de treino e `"y_test"`, que é onde armazenaremos os rótulos de teste `"= train_test_split(X,y)"`.

Esses "Xs" que foram citados nesta linha geralmente são maiúsculos, e utilizamos esta nomenclatura em aprendizado de máquina, pois o "X" maiúsculo, representa o conjunto de atributos que normalmente é uma matriz e na matemática, é utilizado letras maiúsculas para representar matrizes.

E o "y" é minúsculo, já que ele é um vetor de rótulos. Então, executando esta linha teremos separado o conjunto de treino e teste, onde, por padrão é dividido em 75% dos dados para treino e 25% para teste. E esta técnica de "train-test split" também divide aleatoriamente os dados. Ela embaralha os dois parâmetros e faz a divisão em treino e teste de acordo com a proporção dita anteriormente. Dessa maneira, já temos os dados separados e iremos voltar para o modelo para treiná-lo e avaliá-lo.

O treinamento deste modelo é feito de uma maneira bem simples. Vocês perceberão

que a maioria dos modelos que utilizam o sklearn são executados da mesma maneira. Iremos utilizar KNN, que é a variável na qual armazenamos o modelo. Ficando: `knn.fit(X_train,y_train)`.

Ao executar esta linha, teremos armazenado esses valores que são os valores de treino, no KNN. E para avaliar esse modelo iremos utilizar: `knn.score(X_test,y_test)`.

Ao executar esta linha, gerou-se o resultado com o valor 0,6. Todas as vezes que essa técnica for executada normalmente ela dará resultados diferentes próximos, porém diferentes porque a divisão em treino e teste divide os dados de maneira aleatória caso isso não tenha sido dito ou explicitado anteriormente.

Neste caso, estamos utilizando aleatório até para não correremos nenhum risco de viés nos dados.

Então, este modelo acertou 0,6%. Não é um acerto muito alto é um acerto bem baixo, na realidade.

Normalmente, consideramos um bom acerto acima de 70% ou 80%, mas já é um primeiro resultado de um modelo treinado no conjunto de dados.

Com isso, um dos pontos que deve ser considerado no KNN é que, a ordem de grandeza dos dados vai fazer a diferença na distância. Por exemplo, nós temos uma coluna onde a massa vai desde elementos com poucas centenas até três centenas. Ou seja, o menor elemento tem cerca de 140 unidades de massa e o maior elemento tem mais de 300.

Enquanto na cor, a menor cor é 0.55 e a maior é 0.93.

As grandezas estão bem diferentes entre si. No cálculo da distância, a grandeza maior irá influenciá-la muito mais do que a grandeza menor.

E desse jeito, estamos dando prioridades diferentes para colunas diferentes nos dados.

E, por isso, o nosso resultado pode estar sendo ruim. Para tentar balancear esta diferença podemos utilizar a técnica de escalar os dados para obter resultados melhores.

Iremos utilizar a escala Min e Max para tentar melhorar o resultado deste modelo.

Iremos trabalhar com esta escala no próximo vídeo.

Até lá!